



PANDAS

The Complete Confusion Buster Guide

Filtering · groupby · count · max · nlargest · Row Retrieval · Missing Data · Anomalies · Normalization

Everything you were confused about — explained simply, with analogies, diagrams, and real code.

□ Table of Contents

PART A — Core Operations

- [1. The Master Decision Map](#)
- [2. Boolean Filtering — `df\[df\[col\] == value\]`
 - \[2.1 What is it & how it works\]\(#\)
 - \[2.2 Single condition\]\(#\)
 - \[2.3 Multiple conditions \\(AND / OR / NOT\\)\]\(#\)
 - \[2.4 Boolean Filter vs `.loc\\[ \\]`\]\(#\)](#)
- [3. `groupby` — Group & Summarize Data](#)
 - [3.1 What is `groupby`?](#)
 - [3.2 Basic `groupby` + aggregation](#)
 - [3.3 Multiple columns](#)
 - [3.4 `.agg\(\)` — many stats at once](#)
- [4. Boolean Filtering vs `groupby` — When to Use Each](#)
- [5. `.count\(\)` vs `.value\_counts\(\)`](#)
- [6. `max\(\)` vs `idxmax\(\)`](#)
- [7. `nlargest\(\)` and `nsmallest\(\)`](#)
- [8. Quick Stats — sum, mean, median, std](#)
- [9. `.apply\(\)` — Custom Functions](#)

PART B — Row Retrieval

- [10. Returning Rows — `.loc`, `.iloc`, and More](#)
 - [10.1 Return one specific row by index](#)
 - [10.2 Return a row by label with `.loc\[ \]`](#)
 - [10.3 Return a row by position with `.iloc\[ \]`](#)
 - [10.4 Return the full row of a `max/min/idxmax` result](#)
 - [10.5 Return multiple rows \(slicing\)](#)
 - [10.6 `.loc` vs `.iloc` — side-by-side](#)

PART C — Data Cleaning

- [11. Handling Missing \(Null\) Values](#)
 - [11.1 Detecting missing values](#)
 - [11.2 Dropping missing values \(`.dropna`\)](#)
 - [11.3 Filling missing values \(`.fillna`\)](#)
 - [11.4 Filling with mean / median / mode](#)
 - [11.5 Forward fill & backward fill](#)
 - [11.6 Interpolation](#)
 - [11.7 Which strategy to choose?](#)

PART D — Anomalies & Outliers

- [12. Detecting & Handling Anomalies / Outliers](#)

[12.1 What is an anomaly / outlier?](#)

[12.2 The IQR method](#)

[12.3 The Z-Score method](#)

[12.4 Visual detection](#)

[12.5 Handling anomalies — what to do with them](#)

[12.6 Duplicate rows](#)

[12.7 Wrong data types](#)

PART E — Normalization & Scaling

[13. Normalization & Scaling](#)

[13.1 Why normalize?](#)

[13.2 Min-Max Normalization \(0 to 1\)](#)

[13.3 Standardization / Z-Score Scaling](#)

[13.4 Robust Scaling \(for outlier-heavy data\)](#)

[13.5 Log Transformation](#)

[13.6 Which scaling method to choose?](#)

[14. Master Cheat Sheet — Every Tool at a Glance](#)

PART A — Core Operations

PART A — Core Operations

1. The Master Decision Map

Before we learn anything else, here is the map that answers: 'Which tool should I use?'

□ The Master Decision Map	
What do you want to do?	
KEEP certain rows only (Filter / WHERE)	<code>df[df['col'] == value]</code> → same number of columns, fewer rows
SUMMARIZE per category (totals, averages, etc.)	<code>df.groupby('col').agg(...)</code> → one row per unique category
GET a specific row	<code>df.loc[label]</code> or <code>df.iloc[position]</code>
COUNT items per category COUNT non-null values	<code>.value_counts()</code> <code>.count()</code>
HIGHEST / LOWEST value WHICH row has max/min? FULL ROW of max/min?	<code>.max()</code> / <code>.min()</code> <code>.idxmax()</code> / <code>.idxmin()</code> <code>df.loc[df['col'].idxmax()]</code>
TOP N rows	<code>df.nlargest(n, 'col')</code>
FIX missing values	<code>.fillna()</code> or <code>.dropna()</code>
REMOVE outliers	IQR method or Z-Score method
SCALE / NORMALIZE data	Min-Max / Z-Score / Log Transform

2. Boolean Filtering — `df[df[col] == value]`

2.1 What is it & how it works

Boolean filtering means **keeping only the rows where a condition is True**. Every row is tested — rows that pass stay in; rows that fail are removed.

□ Analogy: Airport Security Scanner

Imagine a conveyor belt at an airport. Every bag (row) goes through the scanner (your condition). If the bag passes — it comes through. If it fails — it is stopped. Boolean filtering is that scanner: every row is checked, only passing rows continue.

□ How Boolean Filtering Works Step by Step

```
df [ df['PROVINCE'] == 'PUNJAB' ]
```

↑	↑	
		Inner part: Creates a Series of True/False
	0	True ← this row is Punjab
	1	False ← this row is Sindh
	2	True ← this row is Punjab
	3	False ← this row is KPK
	...	

Outer `df[...]`: Keeps only the rows where value is True
 Result: a new DataFrame with ONLY Punjab rows

2.2 Single Condition

```
# Equal to
punjab = df[ df['PROVINCE'] == 'PUNJAB' ]

# Greater than
big_cities = df[ df['POPULATION'] > 1_000_000 ]

# Greater than or equal to
df[ df['POPULATION'] >= 500_000 ]

# Not equal to (exclude a value)
df[ df['PROVINCE'] != 'BALOCHISTAN' ]

# Text contains (partial match)
df[ df['CITY'].str.contains('abad') ] # finds Islamabad, Hyderabad, Faisalabad...

# Text starts with
df[ df['CITY'].str.startswith('K') ]
```

2.3 Multiple Conditions (AND / OR / NOT)

CRITICAL: **Each condition MUST be in its own parentheses**. Forgetting this is the #1 beginner mistake.

```
# AND ( & ) - BOTH conditions must be True
result = df[ (df['PROVINCE'] == 'PUNJAB') & (df['POPULATION'] > 500_000) ]

# OR ( | ) - AT LEAST ONE condition must be True
result = df[ (df['PROVINCE'] == 'PUNJAB') | (df['PROVINCE'] == 'SINDH') ]

# NOT ( ~ ) - Flips True to False and False to True
result = df[ ~(df['PROVINCE'] == 'PUNJAB') ] # everything EXCEPT Punjab

# Shortcut for OR with many values - .isin() is cleaner
result = df[ df['PROVINCE'].isin(['PUNJAB', 'SINDH', 'KPK']) ]

# Combining 3+ conditions
result = df[ (df['PROVINCE'] == 'PUNJAB') & (df['POPULATION'] > 100_000) &
(df['YEAR'] == 2023) ]
```

□ Common Mistake: Forgetting parentheses around each condition

WRONG: `df[df['A'] == 1 & df['B'] == 2]` ← Python sees '1 & df[...]' first — error! CORRECT:
`df[(df['A'] == 1) & (df['B'] == 2)]` ← each condition in its own ()

2.4 Boolean Filter vs .loc[]

Feature	Boolean Filter <code>df[cond]</code>	<code>.loc[cond, cols]</code>
Purpose	Filter rows only	Filter rows AND select columns at once
Syntax complexity	Simpler	Slightly more typing
Select columns?	Need a second step	Yes, right inside <code>.loc</code>
Example	<code>df[df['A'] > 5]</code>	<code>df.loc[df['A'] > 5, ['A', 'B']]</code>
Which is faster?	Same speed	Same speed
Beginner friendly?	✓Yes, start here	Learn after basics

3. groupby — Group & Summarize Data

3.1 What is groupby?

groupby splits the DataFrame into groups, then **applies a calculation to each group**. It is the Pandas version of a PivotTable or SQL GROUP BY.

□ Analogy: Sorting Mail by City

A postman has 200 letters going to 5 cities. First, he SORTS (groups) letters by city into separate piles. Then he COUNTS each pile. That is groupby: split into groups, then calculate something about each group.

□ How groupby Works — Visualized

Original Data (8 rows):		After groupby('Category')['Sales'].sum():	
Category	Sales	Category	Sales
A	100	A	370
B	200	B	680
A	150	C	450
C	250		
A	120		
B	180		
C	200		
B	300		

STEP 1: Group into piles: {A: rows 0,2,4}, {B: rows 1,5,7}, {C: rows 3,6}

STEP 2: Sum each pile

STEP 3: Return one row per group

← 100+150+120
← 200+180+300
← 250+200

3.2 Basic groupby + Aggregation

```
# Basic pattern: df.groupby('GROUP_COL')['VALUE_COL'].AGGREGATION()

df.groupby('Category')['Sales'].sum()      # total per category
df.groupby('Category')['Sales'].mean()     # average per category
df.groupby('Category')['Sales'].max()      # highest sale per category
df.groupby('Category')['Sales'].min()      # lowest sale per category
df.groupby('Category')['Sales'].count()    # how many rows per category
df.groupby('Category')['Sales'].std()      # standard deviation per category

# Loop to 'see' the groups (for understanding only):
for group_name, group_df in df.groupby('Category'):
    print(group_name)    # 'A', 'B', 'C'
    print(group_df)      # the rows belonging to that group
```

3.3 Multiple Columns

```
# Group by two columns — creates a hierarchy
result = df.groupby(['Store', 'Category'])['Sales'].sum()

# Output has a multi-level index (Store → Category):
# Store  Category
```



```
# S1      A      220
#         B      500
# S2      A      150
#         B      180
#         C      450

# Flatten back to a regular table:
result = result.reset_index()
```

3.4 .agg() — Many Stats at Once

```
# Run multiple aggregations in one go
result = df.groupby('Category')['Sales'].agg(['sum', 'mean', 'max', 'min', 'count'])

# With custom column names (named aggregation):
result = df.groupby('Category').agg(
    total_sales = ('Sales', 'sum'),
    avg_sales   = ('Sales', 'mean'),
    max_sale    = ('Sales', 'max'),
    total_qty   = ('Quantity', 'sum'),
    order_count = ('Sales', 'count'),
)
```

4. Boolean Filtering vs groupby — When to Use Each

✓ Rule: The Golden Rule

Boolean Filtering → result has the SAME number of rows (you removed some, but each row is still one original row) groupby → result has FEWER rows (one row per unique group — the data is summarized/collapsed)

Question You Are Asking	Right Tool	Code
Show me only Punjab rows	Boolean Filter	<code>df[df['PROVINCE']=='PUNJAB']</code>
Total population per province	groupby + sum	<code>df.groupby('PROVINCE')['POP'].sum()</code>
Sales where amount > 1000	Boolean Filter	<code>df[df['Sales'] > 1000]</code>
Average sales per store	groupby + mean	<code>df.groupby('Store')['Sales'].mean()</code>
Rows in Karachi in 2023	Boolean Filter	<code>df[(df['City']=='Karachi') & (df['Year']==2023)]</code>
Orders per category	groupby + count	<code>df.groupby('Category')['Sales'].count()</code>
Remove zero-population rows	Boolean Filter	<code>df[df['POP'] != 0]</code>
Best-selling category name	groupby + idxmax	<code>df.groupby('Cat')['Sales'].sum().idxmax()</code>

5. .count() vs .value_counts()

□ Analogy: Attendance Register

SCENARIO: 30 students, some didn't fill in their city. `.count()` → 'How many students filled in a city at all?' → gives ONE number (e.g., 27) `.value_counts()` → 'How many students are from Karachi, how many from Lahore, etc.?' → gives a frequency table

□ count() vs value_counts() on the Same Column

Column: PROVINCE

0	PUNJAB	<code>.count()</code>	→ 6	(counts non-null entries)
1	SINDH			
2	PUNJAB	<code>.value_counts()</code>	→	
3	NaN		PUNJAB	3
4	KPK		SINDH	2
5	SINDH		KPK	1
6	PUNJAB			

NaN is NOT counted by either (by default)

```
# .count() - how many non-null values in each column
df['PROVINCE'].count()    # → 6    (a single number)
df.count()                # → counts non-null per EVERY column at once

# .value_counts() - frequency of each unique value
df['PROVINCE'].value_counts()    # sorted: highest count first
df['PROVINCE'].value_counts(normalize=True)    # show as % instead of counts
df['PROVINCE'].value_counts(ascending=True)    # lowest count first
df['PROVINCE'].value_counts(dropna=False)    # also count NaN rows
```

Feature	<code>.count()</code>	<code>.value_counts()</code>
Returns	A single integer (or one per column)	A Series: value → frequency
Use for	Checking for missing data	Understanding distribution
Sorted?	No	Yes — highest first
Handles NaN?	Excludes them (counts non-null)	Excludes them by default
Works on whole df?	✓ <code>df.count()</code>	✗ Use on one column at a time

6. max() vs idxmax()

□ Analogy: Exam Results Board

max() → 'The highest mark in the class was 98.' idxmax() → 'The student who scored highest is Ahmed (row 4).' max() gives you the VALUE. idxmax() gives you the LABEL/INDEX (who or what has that value).

□ max() vs idxmax()

Series with default index:

idx	Sales
0	100
1	300
2	150

← max

Series with city index:

City	Population
Lahore	11,126,285
Karachi	9,340,000
Quetta	1,001,205

← max

.max() = 300
.idxmax() = 1

.max() = 11,126,285
.idxmax() = 'Lahore'

```
df['Sales'].max()      # → 300          (the value itself)
df['Sales'].idxmax()   # → 1           (the INDEX where that value lives)
df['Sales'].min()      # → 100
df['Sales'].idxmin()   # → 0

# Getting the FULL ROW of the maximum – see Chapter 10 for more on this
max_idx = df['Sales'].idxmax() # step 1: find the index
df.loc[max_idx]               # step 2: retrieve that full row

# One-liner version:
df.loc[ df['Sales'].idxmax() ] # full row of the max Sale
```

Method	Returns	Example output
.max()	The maximum VALUE (a number)	300
.idxmax()	The INDEX/LABEL of the row with max value	1 or 'Lahore'
.min()	The minimum VALUE	100
.idxmin()	The INDEX/LABEL of the row with min value	0 or 'Quetta'

7. nlargest() and nsmallest()

`nlargest(n, col)` **returns the top N rows** with the highest values in a column. It is `.sort_values() + .head()` in one step — but faster.

□ Analogy: The Leaderboard

In a game tournament, you want to display the Top 10 players. You don't want ALL players — just the best 10. `nlargest(10, 'score')` does exactly that. `nsmallest(10, 'score')` gives you the 10 who struggled most.

```
# Top 5 most populated cities
df.nlargest(5, 'POPULATION')

# Bottom 5 least populated
df.nsmallest(5, 'POPULATION')

# On a Series (single column):
df['Sales'].nlargest(3)      # top 3 values
df['Sales'].nsmallest(3)     # bottom 3 values

# Equivalent (but longer) using sort_values:
df.sort_values('POPULATION', ascending=False).head(5)  # same as nlargest
df.sort_values('POPULATION', ascending=True).head(5)   # same as nsmallest
```

8. Quick Stats — sum, mean, median, std

□ Quick Stats Reference		
Data: [100, 200, 300, 400, 500]		
.sum()	→ 1500.0	(100+200+300+400+500)
.mean()	→ 300.0	(1500 ÷ 5 = average)
.median()	→ 300.0	(middle value when sorted)
.min()	→ 100.0	(smallest value)
.max()	→ 500.0	(largest value)
.std()	→ 158.1	(how spread out the values are)
.count()	→ 5	(how many non-null values)
.var()	→ 25000.0	(variance = std squared)
.describe() shows all 8 at once:		
count	5.000000	
mean	300.000000	
std	158.113883	
min	100.000000	
25%	200.000000	
50%	300.000000	← this is the median
75%	400.000000	
max	500.000000	

✓ Rule: Mean vs Median — Use Median When You Have Outliers

Example salaries: [20,000 / 22,000 / 21,000 / 25,000 / 500,000] Mean = 117,600 ← misleading! Dragged up by the CEO's 500,000 salary Median = 22,000 ← more representative of what a typical employee earns Rule: If your data has extreme outliers, prefer median over mean.

9. .apply() — Custom Functions

□ Analogy: The Factory Assembly Line

`apply()` is a conveyor belt. Every item (value) in your column goes through your custom machine (function), and the result comes out the other side. Perfect when no built-in function does exactly what you need.

```
# Using a named function
def to_millions(x):
    return round(x / 1_000_000, 2)

df['Pop Millions'] = df['POPULATION'].apply(to_millions)

# Using a lambda (one-liner anonymous function)
df['Pop_Millions'] = df['POPULATION'].apply(lambda x: round(x / 1_000_000, 2))

# Cleaning text
df['PROVINCE'] = df['PROVINCE'].apply(lambda x: x.title()) # 'PUNJAB' → 'Punjab'

# Conditional logic
df['Size'] = df['POPULATION'].apply(lambda x: 'Big' if x > 1_000_000 else 'Small')

# Apply across columns (axis=1 means 'apply to each row')
df['Total'] = df.apply(lambda row: row['Urban'] + row['Rural'], axis=1)
```

PART B — Row Retrieval

10. Returning Rows — .loc, .iloc, and More

This chapter answers the question: 'I found something interesting — how do I get the FULL ROW back?'

□ Row Retrieval Quick Reference

HOW TO GET A ROW — Overview

<code>df.loc[label]</code>	→ get row by its INDEX LABEL
<code>df.iloc[position]</code>	→ get row by its POSITION NUMBER (0, 1, 2...)
<code>df.loc[df['col'].idxmax()]</code>	→ get the FULL ROW of the maximum value
<code>df[df['col']==val]</code>	→ get ALL rows where condition is True
<code>df[df['col']==val].iloc[0]</code>	→ get the FIRST matching row

10.1 Return One Specific Row by Index Number

The most common case: you know the row number and want everything in that row.

```
# Get row at position 0 (first row)
df.iloc[0]          # returns a Series — index=column names, values=row data

# Get row at position 5 (sixth row)
df.iloc[5]

# Get the LAST row
df.iloc[-1]         # negative index = count from the end

# Get the second-to-last row
df.iloc[-2]

# To get it as a DataFrame (table format) instead of a Series:
df.iloc[[0]]        # double brackets → returns a 1-row DataFrame
df.iloc[[5]]
```

□ Tip: Series vs DataFrame return

`df.iloc[0]` → returns a Series (column names as index, one value per column) `df.iloc[[0]]` → returns a 1-row DataFrame (same shape as original, just one row) Use `[[ ]]` (double brackets) when you want the result to still look like a table.

10.2 Return a Row by Label with .loc[]

`.loc` uses the INDEX LABEL (not position number). *If your index is 0,1,2,3... then label = position. But if you set a custom index (like city names), label ≠ position.*

```
# Default numeric index — .loc[n] same as .iloc[n]
df.loc[0]          # row with label '0'
df.loc[5]          # row with label '5'
```

```
# Custom index (e.g., after set_index):
df2 = df.set_index('CITY')
df2.loc['Lahore']      # row where CITY == 'Lahore'
df2.loc['Karachi']     # row where CITY == 'Karachi'

# .loc also works with conditions (boolean mask):
df.loc[df['PROVINCE'] == 'PUNJAB']      # all Punjab rows

# .loc to get rows AND specific columns at the same time:
df.loc[df['PROVINCE'] == 'PUNJAB', ['DISTRICT', 'POPULATION']]
#                                     ↑ only these columns
```

10.3 Return a Row by Position with .iloc[]

.iloc always uses **integer position (0, 1, 2...)** — regardless of what the actual index labels are. Think of it like a numbered list.

□ .iloc uses Position, .loc uses Label

df with custom index:

Index	City	Population	
			.iloc[0] = row with Lahore
			.iloc[1] = row with Karachi
			.iloc[2] = row with Quetta
100	Lahore	11,126,285	
200	Karachi	9,340,000	.loc[100] = row with Lahore
300	Quetta	1,001,205	.loc[200] = row with Karachi

Even though labels are 100/200/300,
iloc still counts 0/1/2 by POSITION.

```
df.iloc[0]      # first row (position 0)
df.iloc[10]     # 11th row (position 10)
df.iloc[-1]     # last row

# Slicing rows by position:
df.iloc[0:5]    # rows at positions 0,1,2,3,4 (NOT 5)
df.iloc[10:20]  # rows at positions 10 through 19

# Rows AND columns by position:
df.iloc[0, 0]   # value at row 0, column 0
df.iloc[0, 2]   # value at row 0, column 2
df.iloc[0:5, 0:3] # first 5 rows, first 3 columns
```

10.4 Return the Full Row of a max / min / idxmax Result

This is the most common real-world row-retrieval task: 'Show me everything about the city with the highest population.'

```
# PATTERN: df.loc[ df['col'].idxmax() ]

# Get the FULL ROW of the maximum population city:
idx = df['POPULATION'].idxmax()  # step 1: find the row label
df.loc[idx]                      # step 2: retrieve that row
```

```
# One-liner (most common way):
df.loc[ df['POPULATION'].idxmax() ]

# Full row of the MINIMUM:
df.loc[ df['POPULATION'].idxmin() ]

# Full row of nlargest — already returns full rows!
df.nlargest(1, 'POPULATION')      # top 1 row (as a DataFrame)

# Full row of the max WITHIN a group (after groupby):
# Which city in each province has the highest population?
df.loc[ df.groupby('PROVINCE')['POPULATION'].idxmax() ]
```

□ What df.loc[df['col'].idxmax()] Returns

```
df['POPULATION'].idxmax() → returns 42 (the row label)
df.loc[42]                → returns the full row 42:
```

PROVINCE	PUNJAB
DISTRICT	Lahore
TEHSIL	Lahore City
POPULATION	11126285
ALL SEXES (RURAL)	985432
ALL SEXES (URBAN)	10140853
...	
Name: 42, dtype: object	

10.5 Returning Multiple Rows (Slicing)

```
# Rows 0 to 4 (5 rows total)
df.iloc[0:5]

# Rows 10 to 19
df.iloc[10:20]

# Every other row (step=2)
df.iloc[::2]

# Last 10 rows
df.iloc[-10:]

# Using .loc with a list of labels:
df.loc[[0, 5, 10, 15]]    # rows at labels 0, 5, 10, 15

# First 5 + Last 5 combined (as in your notebooks):
import pandas as pd
pd.concat([df.head(5), df.tail(5)])
```

10.6 .loc vs .iloc — Side-by-Side

Feature	.loc[label]	.iloc[position]
What it uses	Index LABEL (could be text or number)	Integer POSITION (always 0,1,2,...)
Slicing includes end?	YES — df.loc[0:5] includes row 5	NO — df.iloc[0:5] stops before 5

Feature	.loc[label]	.iloc[position]
Works with text index?	✓df.loc['Lahore']	✓df.iloc[0] (still works by position)
Get row AND columns?	df.loc[cond, ['A','B']]	df.iloc[0, 0:3]
Filter with condition?	✓df.loc[df['A']>5]	✗Must use boolean filter separately
Best for	Label-based access, filtering	Position-based slicing

□ **Common Mistake: .loc slicing INCLUDES the end, .iloc does NOT**

df.loc[0:5] → returns rows 0, 1, 2, 3, 4, 5 (6 rows — includes 5) df.iloc[0:5] → returns rows 0, 1, 2, 3, 4 (5 rows — stops before 5) This is one of the most confusing differences between the two!

PART C — Data Cleaning

11. Handling Missing (Null) Values

Missing values in Pandas are represented as NaN (Not a Number). They appear when data is absent, not recorded, or corrupted. Handling them correctly is one of the most important data cleaning skills.

□ Analogy: The Incomplete Survey Form

Imagine 500 people filled out a survey but some left questions blank. Those blank answers are NaN in Pandas. You have choices: 1. IGNORE rows with blanks (dropna) 2. FILL blanks with a reasonable guess (fillna with mean/median/mode) 3. ESTIMATE from neighboring values (forward fill / interpolation) Which approach to choose depends on WHY the data is missing and how much is missing.

11.1 Detecting Missing Values

```
# Check if each value is NaN (True/False table)
df.isnull()          # True where value is NaN
df.isna()            # same as isnull() - two names, one function

# Count missing values per column
df.isnull().sum()    # most useful - shows how many NaN per column

# As a percentage
df.isnull().mean() * 100  # % of missing values per column

# Check a single column
df['POPULATION'].isnull().sum()  # how many NaN in population column

# Find which rows have ANY missing value
df[ df.isnull().any(axis=1) ]    # rows where at least one column is NaN

# Quick overview (info also shows non-null counts):
df.info()

# If a column shows '480 non-null' and df has 528 rows → 48 values are missing
```

□ Reading Missing Value Summary

df.isnull().sum() output example:

PROVINCE	0	← no missing
DISTRICT	0	← no missing
TEHSIL	0	← no missing
ALL SEXES (RURAL)	8	← 8 missing values!
ALL SEXES (URBAN)	12	← 12 missing values!
MALE (RURAL)	8	
FEMALE (RURAL)	8	
dtype: int64		

Rule of thumb:

<5% missing → safe to drop or fill
 5-30% missing → fill carefully (mean/median/mode)
 >30% missing → consider dropping the whole column

11.2 Dropping Missing Values (.dropna)

dropna() removes rows (or columns) that contain NaN. Use it when missing data is rare and the rows are not important.

```
# Drop ANY row that has at least one NaN
df_clean = df.dropna()

# Drop rows where a SPECIFIC column is NaN
df_clean = df.dropna(subset=['POPULATION'])
df_clean = df.dropna(subset=['POPULATION', 'ALL SEXES (URBAN)']) # any of these

# Drop only if ALL values in the row are NaN (very strict)
df_clean = df.dropna(how='all')

# Drop columns that have any NaN (axis=1 means columns)
df_clean = df.dropna(axis=1)

# Drop columns that have more than 50% NaN
threshold = len(df) * 0.5
df_clean = df.dropna(axis=1, thresh=int(threshold))

# IMPORTANT: dropna() returns a NEW DataFrame — original is unchanged
# Use inplace=True to modify the original:
df.dropna(inplace=True)
```

□ Common Mistake: dropna() loses data!

If you have 528 rows and drop all rows with any NaN, you might end up with 450 rows. That means 78 rows (15% of your data!) were thrown away. Always check: `df.shape` before and after `dropna()` to see how much you lost.

11.3 Filling Missing Values (.fillna)

.fillna() replaces NaN with a value you choose. This is better than `dropna()` when you don't want to lose data.

```
# Fill all NaN with a fixed value (e.g., 0)
df['POPULATION'].fillna(0)

# Fill NaN with the string 'Unknown'
df['PROVINCE'].fillna('Unknown')

# Fill different columns with different values:
df.fillna({'POPULATION': 0, 'PROVINCE': 'Unknown', 'YEAR': 2017})

# To save the result:
df['POPULATION'] = df['POPULATION'].fillna(0)
# OR
df.fillna(0, inplace=True) # modifies df directly
```

11.4 Filling with Mean / Median / Mode

This is the most common and smartest way to fill missing numeric data. Instead of putting 0 (which is often wrong), you put the average value.

```
# Fill with MEAN (average) – good for normally distributed data
mean_val = df['POPULATION'].mean()
df['POPULATION'] = df['POPULATION'].fillna(mean_val)

# One-liner:
df['POPULATION'] = df['POPULATION'].fillna(df['POPULATION'].mean())

# Fill with MEDIAN – better when data has outliers
df['POPULATION'] = df['POPULATION'].fillna(df['POPULATION'].median())

# Fill with MODE (most common value) – best for categorical/text columns
mode_val = df['PROVINCE'].mode()[0] # [0] because mode() returns a Series
df['PROVINCE'] = df['PROVINCE'].fillna(mode_val)

# Fill different types appropriately:
for col in df.select_dtypes(include='number').columns: # numeric columns
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include='object').columns: # text columns
    df[col] = df[col].fillna(df[col].mode()[0])
```

Fill Strategy	Use When	Example
fillna(0)	Zero is a valid/meaningful value	Sales column: no sale = 0
fillna(mean)	Data is roughly symmetric (bell-shaped)	Heights, test scores
fillna(median)	Data has outliers (skewed)	Salaries, population
fillna(mode)	Categorical / text column	Province, Category, Gender
fillna('Unknown')	Text column, no obvious default	City name is missing

11.5 Forward Fill & Backward Fill

Perfect for time-series data (prices, sensor readings, stock data) where missing values should logically be the same as the previous or next recorded value.

□ Forward Fill on Time Series Data			
Date	Temperature		
2023-01-01	25.0		
2023-01-02	NaN	← forward fill →	25.0 (copies from above)
2023-01-03	NaN	← forward fill →	25.0 (copies from above)
2023-01-04	28.0		
2023-01-05	NaN	← forward fill →	28.0
Backward fill would go the other direction (from below).			

```
# Forward fill – copies the last VALID value downward
```



```
df['Temperature'].ffill()          # fills NaN with value above
df['Temperature'].fillna(method='ffill') # older syntax (same result)

# Backward fill — copies the next VALID value upward
df['Temperature'].bfill()          # fills NaN with value below
df['Temperature'].fillna(method='bfill')

# Limit how many consecutive NaNs to fill:
df['Temperature'].ffill(limit=2)   # fill at most 2 consecutive NaNs
```

11.6 Interpolation

Interpolation estimates missing values by calculating a value between surrounding data points. It is more sophisticated than forward/backward fill.

□ Linear Interpolation — Filling Between Known Points

idx	Value
0	10
1	NaN ← linear interpolation → 20 (halfway between 10 and 30)
2	30
3	NaN ← linear interpolation → 40 (halfway between 30 and 50)
4	50

```
# Linear interpolation (default) — draws a straight line between known points
df['Value'].interpolate()

# Polynomial interpolation (curves instead of straight lines)
df['Value'].interpolate(method='polynomial', order=2)

# Time-based interpolation (for datetime index)
df['Value'].interpolate(method='time')
```

11.7 Which Strategy to Choose?

Situation	Best Strategy	Why
Very little missing data (<2%)	dropna()	Losing a few rows is fine
Lots of missing data in numeric col	fillna(median)	Median is robust to outliers
Normally distributed numeric data	fillna(mean)	Mean captures typical value
Categorical / text column	fillna(mode)	Most common value is a safe guess
Time series / ordered data	ffill() or bfill()	Carry forward last known value
Smooth trend data	interpolate()	Estimate from neighbors
Feature for ML model	fillna(median) or model	Never leave NaN — models crash
>30% of column is missing	Drop the column entirely	Too much guessing is unreliable

PART D — Anomalies & Outliers

12. Detecting & Handling Anomalies / Outliers

12.1 What is an Anomaly / Outlier?

An outlier is a data point that is far away from the rest of the data. It could be a genuine extreme value, a data entry error, or a sensor glitch.

□ Analogy: The Heights Example

Heights of 30 students: [155, 160, 158, 162, 175, 168, 163, 161, 500, 159, 165] The value '500' is clearly wrong (no human is 500 cm tall). That is an outlier caused by a data entry error. But in sales data, a single order of \$50,000 among mostly \$500 orders might be a genuine VIP customer — not an error. Context matters when deciding what to do with outliers!

□ Types of Anomalies

Types of anomalies you will encounter:

1. GLOBAL OUTLIERS — value is extreme compared to ALL data
Example: Age = 999, Population = -1, Height = 500 cm
2. WRONG DATA TYPES — text where numbers expected, etc.
Example: POPULATION column has value 'N/A' or '--'
3. DUPLICATES — same row recorded twice
Example: same city appears twice with identical values
4. INCONSISTENT FORMAT — 'Punjab', 'PUNJAB', 'punjab' all mean the same
5. IMPOSSIBLE VALUES — negative age, population < 0, percentage > 100

12.2 The IQR Method (Interquartile Range)

The IQR method is the most reliable statistical way to detect outliers. It does not rely on the mean (which itself gets affected by outliers).

□ IQR Method — How It Works

$IQR = Q3 - Q1$ (the middle 50% of your data)

Data sorted: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 500]

↑
Q1=27.5
(25th percentile)

↑
Q3=77.5
(75th percentile)

$IQR = 77.5 - 27.5 = 50$

Lower bound = $Q1 - 1.5 \times IQR = 27.5 - 75 = -47.5$

Upper bound = $Q3 + 1.5 \times IQR = 77.5 + 75 = 152.5$

Any value BELOW -47.5 or ABOVE 152.5 is an OUTLIER

→ 500 is an outlier! 10 and 100 are NOT outliers.

```
col = df['POPULATION']

Q1 = col.quantile(0.25)    # 25th percentile
Q3 = col.quantile(0.75)    # 75th percentile
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f'Lower bound: {lower_bound:,.0f}')
print(f'Upper bound: {upper_bound:,.0f}')

# Find outlier rows:
outliers = df[ (col < lower_bound) | (col > upper_bound) ]
print(f'Outlier rows: {len(outliers)}')
print(outliers)

# Remove outliers (keep only 'normal' rows):
df_clean = df[ (col >= lower_bound) & (col <= upper_bound) ]
```

12.3 The Z-Score Method

Z-Score measures how many standard deviations a value is from the mean. Values with Z-Score > 3 or < -3 are usually considered outliers.

□ Z-Score — Understanding the Scale

Z-Score formula: $z = (\text{value} - \text{mean}) / \text{std_deviation}$

Interpretation:

$z = 0$	→ exactly average
$z = 1$	→ 1 standard deviation above average
$z = -1$	→ 1 standard deviation below average
$z > 3$	→ unusually HIGH — likely outlier
$z < -3$	→ unusually LOW — likely outlier

Rule of thumb: $|z| > 3 = \text{outlier}$ (catches ~99.7% of normal data)

```
# Method 1: Manual Z-score calculation
col = df['POPULATION']
mean = col.mean()
std = col.std()

df['z_score'] = (col - mean) / std

# Find outliers (Z-score beyond ±3):
outliers = df[ df['z_score'].abs() > 3 ]
print(f'Outliers found: {len(outliers)}')

# Remove outliers:
df_clean = df[ df['z_score'].abs() <= 3 ]

# Method 2: Using scipy (more robust)
from scipy import stats
z_scores = stats.zscore(df['POPULATION'].dropna())
outlier_mask = abs(z_scores) > 3
```

Method	Best When	Limitation
IQR Method	Data has outliers that affect the mean	Might be too aggressive for small datasets
Z-Score	Data is roughly normally distributed	Mean itself is affected by outliers
Visual (boxplot)	Exploratory, quick check	Not precise — just for spotting
.describe() check	Quick sanity check on min/max	Doesn't quantify — manual judgment

12.4 Visual Detection

```
import matplotlib.pyplot as plt

# Boxplot — the whiskers show the IQR; dots beyond whiskers = outliers
df['POPULATION'].plot(kind='box')
plt.title('Population Distribution — Outliers appear as dots')
plt.show()

# Histogram — outliers appear as lone bars far from the main group
df['POPULATION'].hist(bins=50)
plt.title('Population Histogram')
plt.show()

# Quick check: just look at describe()
print(df['POPULATION'].describe())
# If max is 100x the 75th percentile → likely an outlier
```

12.5 Handling Anomalies — What to Do With Them

Strategy	How	When to Use
Remove outliers	<code>df_clean = df[col <= upper_bound]</code>	When outlier is clearly an error
Cap/Winsorize	<code>df[col].clip(lower, upper)</code>	Keep the row, but limit extreme value
Replace with NaN	<code>df.loc[col > upper, 'col'] = NaN</code> , then <code>fillna</code>	Treat as missing, then fill
Replace with median	<code>df.loc[col > upper, 'col'] = median</code>	Smooth out the extreme
Keep as-is	No change	Outlier is genuine (VIP customer, record flood)
Log transform	<code>df[col] = np.log1p(df[col])</code>	Compress extreme values for ML

```
import numpy as np

# Strategy 1: Remove rows with outliers
df_clean = df[ df['Sales'] <= upper_bound ]

# Strategy 2: Cap/Winsorize — clip to bounds (outlier stays, value is capped)
df['Sales'] = df['Sales'].clip(lower=lower_bound, upper=upper_bound)
```

```
# Strategy 3: Replace outlier value with median
median = df['Sales'].median()
df.loc[ df['Sales'] > upper_bound, 'Sales' ] = median

# Strategy 4: Log transform (reduces impact of outliers)
df['Sales_log'] = np.log1p(df['Sales']) # log1p = log(1+x) handles zeros
```

12.6 Duplicate Rows

Duplicates are a common anomaly — the same row recorded multiple times due to data import errors or system glitches.

```
# Check how many duplicate rows exist:
df.duplicated().sum() # total number of duplicate rows

# View the duplicate rows:
df[ df.duplicated() ] # shows rows that are exact duplicates
df[ df.duplicated(keep=False) ] # shows ALL copies, not just the 2nd+

# Remove duplicates (keeps first occurrence):
df_clean = df.drop_duplicates()

# Remove duplicates based on specific columns only:
df_clean = df.drop_duplicates(subset=['CITY', 'DISTRICT'])

# Keep LAST occurrence instead of first:
df_clean = df.drop_duplicates(keep='last')
```

12.7 Wrong Data Types

Sometimes a column that should be numbers is stored as text — usually because of stray characters like commas, spaces, or 'N/A' strings.

```
# Check current data types:
df.dtypes
df.info()

# Problem: POPULATION column is stored as 'object' (text) instead of int
# Reason: values like '1,234,567' (with commas) or 'N/A' or '--'

# Fix: clean and convert
df['POPULATION'] = df['POPULATION'].str.replace(',', '') # remove commas
df['POPULATION'] = df['POPULATION'].replace('N/A', None) # replace N/A with NaN
df['POPULATION'] = pd.to_numeric(df['POPULATION'], errors='coerce') # convert
# errors='coerce' turns any unconvertible value into NaN (safe!)

# Fix date columns:
df['DATE'] = pd.to_datetime(df['DATE'], errors='coerce')

# Fix text case inconsistency ('Punjab', 'PUNJAB', 'punjab' → all same):
df['PROVINCE'] = df['PROVINCE'].str.strip().str.upper() # → 'PUNJAB'
df['PROVINCE'] = df['PROVINCE'].str.strip().str.title() # → 'Punjab'

# Fix whitespace in strings:
```

```
df['CITY'] = df['CITY'].str.strip() # removes leading/trailing spaces
```

PART E — Normalization & Scaling

13. Normalization & Scaling

13.1 Why Normalize?

Normalization means transforming your data so all columns are on a similar scale. This is critical for machine learning algorithms that use distances or gradients.

□ Analogy: Comparing Apples in Meters vs Kilometers

Imagine a dataset with two columns:

- Distance: values from 1 to 10,000 (in meters)
- Age: values from 1 to 100 (in years)

A machine learning model looking at 'closeness' would think distance is ALWAYS more important just because its numbers are bigger. That is wrong — the scale is just different. Normalization fixes this by bringing all columns to the same scale (like 0–1), so no column gets unfair importance just because of its units.

□ Why Normalization Matters

BEFORE normalization:

Distance	Age
100	25
5000	50
9000	75
10000	100

→

AFTER Min-Max normalization:

Distance	Age
0.01	0.247
0.50	0.495
0.90	0.743
1.00	1.000

Both columns now on scale 0 to 1. Fair comparison!

13.2 Min-Max Normalization (0 to 1)

Formula: $(\text{value} - \text{min}) / (\text{max} - \text{min}) \rightarrow$ Results always between 0 and 1.

Minimum value $\rightarrow$ 0, Maximum value $\rightarrow$ 1, everything else $\rightarrow$ proportional in between.

□ Min-Max Formula — Step by Step

Formula: $x_{\text{normalized}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$

Example: ages [20, 30, 40, 50]

$x_{\text{min}} = 20, x_{\text{max}} = 50$

age 20 $\rightarrow (20-20)/(50-20) = 0/30 = 0.000$

age 30 $\rightarrow (30-20)/(50-20) = 10/30 = 0.333$

age 40 $\rightarrow (40-20)/(50-20) = 20/30 = 0.667$

age 50 $\rightarrow (50-20)/(50-20) = 30/30 = 1.000$

```
# METHOD 1: Pandas manual (no extra library needed)
col = df['POPULATION']
df['POPULATION_norm'] = (col - col.min()) / (col.max() - col.min())

# Normalize ALL numeric columns at once:
```

```
numeric_cols = df.select_dtypes(include='number').columns
df_normalized = df.copy()
for col in numeric_cols:
    df_normalized[col] = (df[col] - df[col].min()) / (df[col].max() - df[col].min())

# METHOD 2: Using sklearn (more convenient for ML pipelines)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler() # creates the scaler
df[numeric_cols] = scaler.fit_transform(df[numeric_cols]) # applies it

# How to REVERSE normalization (get original values back):
df_original = scaler.inverse_transform(df[numeric_cols])
```

□ Tip: When to use Min-Max

✓When you need values strictly between 0 and 1 ✓For neural networks and image pixel normalization ✓When you know the min/max and they are meaningful ✗Avoid when data has extreme outliers — one huge outlier will compress all other values near 0

13.3 Standardization / Z-Score Scaling

Formula: $(\text{value} - \text{mean}) / \text{std} \rightarrow$ Results centered around 0. Typical range: -3 to +3 (but no hard limit).

Unlike Min-Max, standardization does NOT force values into 0–1. Instead, it centers the data so the mean = 0 and standard deviation = 1.

□ Standardization Formula — Step by Step

Formula: $x_{\text{scaled}} = (x - \text{mean}) / \text{std_deviation}$

Example: ages [20, 30, 40, 50]

mean = 35, std = 12.91

age 20 $\rightarrow (20-35)/12.91 = -1.16$ (1.16 std below average)

age 30 $\rightarrow (30-35)/12.91 = -0.39$ (slightly below average)

age 40 $\rightarrow (40-35)/12.91 = +0.39$ (slightly above average)

age 50 $\rightarrow (50-35)/12.91 = +1.16$ (1.16 std above average)

Negative = below average. Positive = above average.

Z-score of 0 = exactly average.

```
# METHOD 1: Pandas manual
col = df['POPULATION']
df['POPULATION std'] = (col - col.mean()) / col.std()

# METHOD 2: Using sklearn StandardScaler
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

# Check result: mean should be ~0, std should be ~1
print(df['POPULATION_std'].mean()) # ~ 0.0
```

```
print(df['POPULATION_std'].std()) # ≈ 1.0
```

□ Tip: When to use Standardization

✓When your data has outliers (Z-score is less affected than Min-Max) ✓For algorithms that assume normally distributed input (linear regression, PCA, SVM) ✓When you don't know the natural min/max range ✓This is the DEFAULT choice when unsure — it works well in most situations

13.4 Robust Scaling (for outlier-heavy data)

Robust Scaling uses the median and IQR instead of mean and std — so extreme outliers barely affect it.

```
# METHOD 1: Manual robust scaling
col = df['POPULATION']
median = col.median()
Q1 = col.quantile(0.25)
Q3 = col.quantile(0.75)
IQR = Q3 - Q1

df['POPULATION_robust'] = (col - median) / IQR

# METHOD 2: Using sklearn RobustScaler
from sklearn.preprocessing import RobustScaler

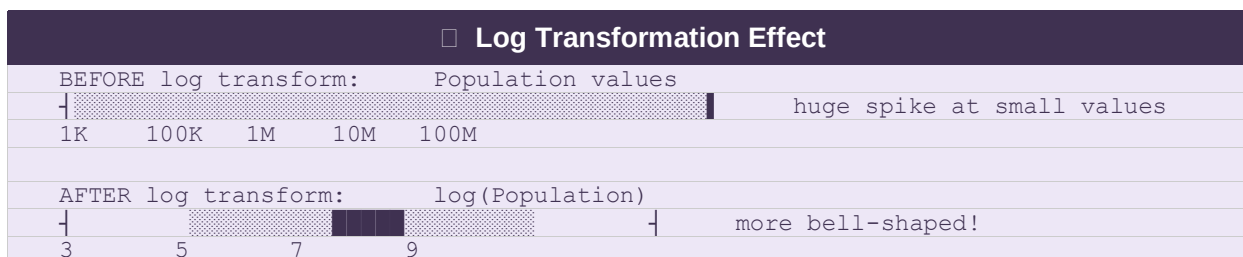
scaler = RobustScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

□ Tip: When to use Robust Scaling

✓When your data has significant outliers that you want to keep ✓When you tried Min-Max or Z-Score and results still seemed skewed ✓Medical data, financial data — where extreme values are real but rare

13.5 Log Transformation

Log transformation is great for highly skewed data (where most values are small but a few are enormous). It compresses the scale and makes the distribution more symmetric.



Use `np.log1p(x)` which computes $\log(1+x)$ — safe for zero values
 Use `np.expml(x)` to reverse: `expml(x)` computes $e^x - 1$

```
import numpy as np

# Log transformation (use log1p for safety — handles zeros)
df['POP_log'] = np.log1p(df['POPULATION'])    # log(1 + x)

# Square root transformation (less aggressive than log)
df['POP_sqrt'] = np.sqrt(df['POPULATION'])

# Reverse log transformation to get original values:
df['POP_original'] = np.expml(df['POP_log'])    # e^x - 1

# Check skewness before and after:
print('Before:', df['POPULATION'].skew())      # high value = very skewed
print('After: ', df['POP_log'].skew())          # should be closer to 0
```

13.6 Which Scaling Method to Choose?

Method	Range	Best When	Handles Outliers?
Min-Max	0 to 1	Need strict 0–1 range; neural networks	✗ Outliers distort scale
Standardization	-3 to +3 (approx)	Most ML algorithms; no obvious range	□ Somewhat affected
Robust Scaling	Centered on median	Data has many outliers to keep	✓ Best for outliers
Log Transform	Varies	Heavily right-skewed data; populations	✓ Compresses extremes
Square Root	Varies	Mildly skewed; count data	□ Partially effective
No scaling	Original	Tree-based models (Random Forest, XGBoost)	N/A — not needed

✓ Rule: The Default Recommendation

When unsure: 1. Start with Standardization (Z-score) — works well in most cases 2. If data is very skewed → try Log Transform first, then Standardization 3. If data has many outliers → use Robust Scaling 4. If building a neural network → Min-Max (0 to 1) 5. If using Random Forest / XGBoost → no scaling needed at all!

14. Master Cheat Sheet — Every Tool at a Glance

Print this page and keep it next to you while coding!

Goal	Tool	Quick Code
FILTERING & SELECTION		
Keep rows where condition is true	Boolean Filter	<code>df[df['col'] == val]</code>
Keep rows — multiple conditions (AND)	Boolean Filter &	<code>df[(df['A']>5) & (df['B']<10)]</code>
Keep rows — multiple conditions (OR)	Boolean Filter	<code>df[(df['A']>5) (df['B']<10)]</code>
Exclude rows	Boolean Filter ~	<code>df[~(df['col'] == val)]</code>
Select rows AND columns	.loc	<code>df.loc[df['A']>5, ['A','B']]</code>
GROUPING & AGGREGATION		
Total per category	groupby + sum	<code>df.groupby('cat')['val'].sum()</code>
Average per category	groupby + mean	<code>df.groupby('cat')['val'].mean()</code>
Count rows per category	groupby + count	<code>df.groupby('cat')['val'].count()</code>
Multiple stats at once	.agg()	<code>df.groupby('cat')['val'].agg([...])</code>
COUNTING		
How many non-null values?	.count()	<code>df['col'].count()</code>
How often does each value appear?	.value_counts()	<code>df['col'].value_counts()</code>
MAX / MIN / TOP N		
Maximum value	.max()	<code>df['col'].max()</code>
Which row has maximum value?	.idxmax()	<code>df['col'].idxmax()</code>
Full row of the maximum	.loc + .idxmax()	<code>df.loc[df['col'].idxmax()]</code>
Top N rows by column	.nlargest()	<code>df.nlargest(5, 'col')</code>
Bottom N rows	.nsmallest()	<code>df.nsmallest(5, 'col')</code>
ROW RETRIEVAL		
Get row by position (0,1,2...)	.iloc[]	<code>df.iloc[0]</code>
Get row by label/index	.loc[]	<code>df.loc[label]</code>
Get multiple rows by position	.iloc[a:b]	<code>df.iloc[0:5]</code>
Full row of max value	.loc[.idxmax()]	<code>df.loc[df['col'].idxmax()]</code>
MISSING VALUES		
Count missing per column	.isnull().sum()	<code>df.isnull().sum()</code>
Drop rows with NaN	.dropna()	<code>df.dropna(subset=['col'])</code>

Goal	Tool	Quick Code
Fill NaN with value	.fillna()	df['col'].fillna(0)
Fill NaN with mean	.fillna(mean)	df['col'].fillna(df['col'].mean())
Fill NaN with median	.fillna(median)	df['col'].fillna(df['col'].median())
Forward fill	.ffill()	df['col'].ffill()
Interpolate	.interpolate()	df['col'].interpolate()
ANOMALIES		
Find outliers (IQR method)	IQR bounds	df[col > Q3 + 1.5*IQR]
Find outliers (Z-Score)	Z > 3	df[(col - mean)/std > 3]
Remove duplicates	.drop_duplicates()	df.drop_duplicates()
Fix data type	pd.to_numeric()	pd.to_numeric(df['col'], errors='coerce')
Cap extreme values	.clip()	df['col'].clip(lower, upper)
NORMALIZATION		
Min-Max (0 to 1)	Manual / MinMaxScaler	(x - x.min())/(x.max() - x.min())
Standardization (Z-score)	StandardScaler	(x - x.mean()) / x.std()
Log transform (skewed data)	np.log1p()	np.log1p(df['col'])
Robust scaling (outliers)	RobustScaler	(x - median) / IQR
Apply custom function	.apply()	df['col'].apply(lambda x: x**2)

□ *Happy Analyzing! Every expert was once a confused beginner.*

When in doubt, go back to Chapter 1 — The Master Decision Map.